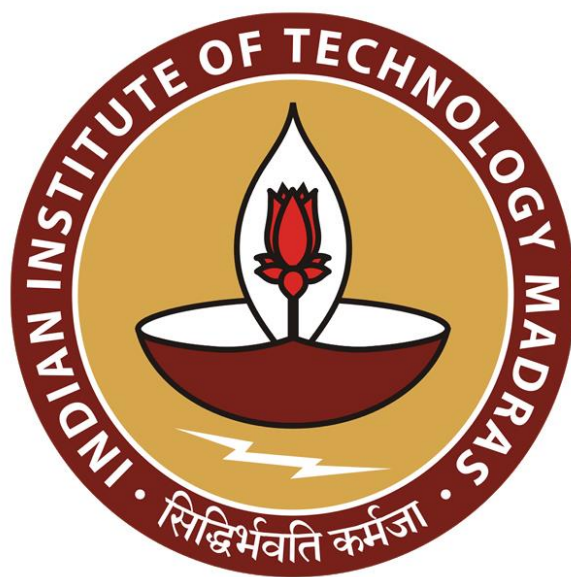


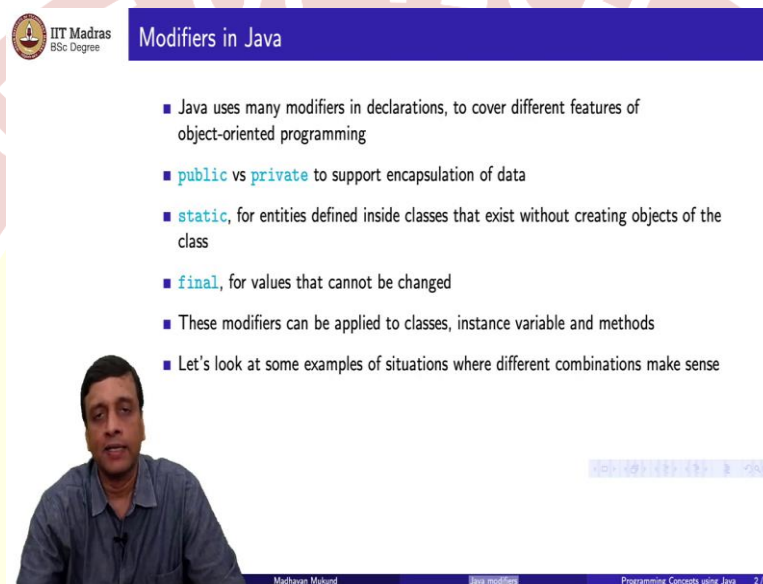


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Java Modifiers

(Refer Slide Time: 00:18)



IT Madras
BSc Degree

Modifiers in Java

- Java uses many modifiers in declarations, to cover different features of object-oriented programming
- **public** vs **private** to support encapsulation of data
- **static**, for entities defined inside classes that exist without creating objects of the class
- **final**, for values that cannot be changed
- These modifiers can be applied to classes, instance variable and methods
- Let's look at some examples of situations where different combinations make sense

Madhavan Mukund

Java modifiers

Programming Concepts using Java 2 / 8

So, to round off this week let us take another look at the various modifiers that we find in java. So, remember that java uses many modifiers because object oriented programming introduces many features connecting the different types that we use in our programs. At the first level because we want to encapsulate the data in our data types we need to distinguish what is private from what is public.

So, the most common modifier that we find is public and private. Then in a very strict object oriented language like java everything has to be defined in the context of a class and remember that normally functions that are defined within a class exist only with respect to an object of the class. So, you have to first create an object and with each object you have associated with it the methods that are defined for that object in that class.

But sometimes we need to create functions and variables that exist without having created objects and so we use the term static for that. And we also want to now allow

some variables to be defined and not modified. So, some kind of constants so we have the term final. So, these modifiers can be applied to classes they can be applied to instance variables they can be applied to methods.

So, let us look at some examples of situations where different combinations of these make sense. So, we have seen some standard combinations. So, let us look at some perhaps non-standard combinations to understand why these things can be used in a meaningful way even in situations which we do not anticipate right up front.

(Refer Slide Time: 01:52)

IIT Madras
BSc Degree

private methods

- Example: a `Stack` class
 - Data stored in a private array
 - Public methods to push, pop, query if empty
- `push()` needs to check if stack has space
- Deal gracefully with stack overflow
 - `private` methods invoked from within `push()` to check if stack is full and expand storage

```
public class Stack {  
    ...  
    public void push (int i){  
        if (stack_full()){  
            extend_stack();  
        }  
        ... // Usual push operations  
    }  
    ...  
    private boolean stack_full(){  
        return (tos == size);  
    }  
    private void extend_stack(){  
        /* Allocate additional space,  
         * reset size etc */  
    }  
}
```

Madhavan Mukund

Java modifiers

Programming Concepts using Java 4 / 8

So, let us begin with public and private. So, as we said public and private is needed for a faithful implementation of encapsulation. So, typically what we would expect is the instance variables that is the variables that define the state of an object are labeled private and then we have public methods which allow us to access this state variables and update them right.

So, we have typically two types of methods we said those which access only. So, the accessor methods they only query the information they do not change anything and then we have those which actually update the information and these we called mutator methods. So, what would we do for instance if there was a private method a private method cannot be called from outside the class.

So, why would anybody want a private method. So, to understand this let us look at a typical example of an abstract data type using a class. So, here is a very simplistic

implementation of a stack. So, we are implementing a stack as an array of values. So, these are all integer values. So, this is an array a stack of integers and we maintain along with this integer array two quantities.

The index of the top of the stack `tos` and also because as we will see this array is going to be dynamic in some sense we will also maintain what is the size of the stack okay. So, we have two quantities in addition to the array we have the index which is natural because this points to the top element of the stack this is where the next update is going to happen and we are also maintaining the size of the stack. So, then we would have we are not showing here some constructors which will set up a stack and so on.

And then the three fundamental operations associated with the stack. So, we have these two mutator methods `push` and `pop`. So, `push` will take an integer and put it at the top of the stack right it will increment the stack size by one and it will put it at the top of the stack and `pop` will tell us what is at the top of the stack and remove it. So, `pop` reduces the content of the stack by one and finally we have a accessor method which just tells us whether this array contains anything or not.

So, it just checks whether the top of the stack is pointing to the initial position. So, the way we have done it the `tos` points to the next position that is going to be filled on the stack. So, when it is the empty stack the top of stack is pointing to index 0. So, the first element that we push onto the stack will go into index 0 and then top of stack will go to 1 right. So, this is a very simple implementation of a stack.

So, what we have to be careful about is what happens for example in `push` right. So, we have an array and an array is of a fixed size. So, clearly there will be a situation where we push a lot of values and the array size is exceeded. So, `push` needs to check if the stack has space right. So, that is why we are using this size right. So, the size is kind of the static limit of how many values we have.

So, essentially this is in if it was an array it is `values.length` right. So, we can think of size as being the length of the current array that we have and. So, long as top of stack is below that remember the top of stack is the place where we are going to put the value

then if it is anything up to the length -1 we can update that position to be the new value we are pushing and then make it the next one right but what happens if we cross right.

So, what happens if we go to the position beyond the length of the stack right. So, we have some kind of a stack overflow. So, we have to deal with it. So, of course one simple way to deal with it is just to prevent or to deny this push to say that this push is now invalid because we have exceeded the content of the stack. But a more graceful way to deal with the stack overflow is to actually expand the stack right.

So, if you want to expand the stack then at this point we will first realize that the stack is full and then we would call a function which will expand the size or give us a larger array to deal with the stack. So, natural way to do that is to have a private function called extend stack right which will for typical strategy in such situations is to double the size of the array. So, supposing we initially add 100 elements in the array and we try to insert 101st element onto the stack.

Then extend stack will typically allocate a 200 size array and copy the contents of the first hundred positions and make that the new stack right. So, this whatever it does we do not care exactly what it does. But the point is that we have to allocate separately new space make sure that the old stack is sitting in the new space and also this variable size which is determining whether or not the stack is full will be updated to be the size of the new space.

And just like we had this empty stack function which told us whether the stack is empty or not now we have a stack full function which checks whether the top of stack has reached the limit. So, now what we do in our push is we first check if we have reached the limit if we have reached the limit the first thing we do is extend the stack. Now once we extend the stack we know that the stack has been faithfully extended in the sense that everything has moved as it is and the top of stack is still where it was. So, we have just moved to a larger array.

So, then we can go ahead and just make the current value put the current value at the current position and increment the top of stack. So, the point is that these functions are called from inside the class right. So, these are private functions there is no reason for an

outside object to know the implementation of the stack and to know that it has been extended. So, it will happen implicitly and it is needed inside the stack to maintain this graceful handling of the overflow but it should not be public.

So, this is a very natural case where you have private methods and we will see more examples as we go along. But it is not useless the point is it is not useless to have private functions they can be usefully put to use inside the other methods which are public in the stack.

(Refer Slide Time: 07:51)

Accessor and mutator methods

- Public methods to query and update private instance variables
- **Date class**
 - Private instance variables **day, month, year**
 - One public accessor/mutator method per instance variable
 - Inconsistent updates are now possible
 - Separately set invalid combinations of **day** and **month**

```
public class Date {  
    private int day, month, year;  
  
    public void getDay(int d) {...}  
    public void getMonth(int m) {...}  
    public void getYear(int y) {...}  
  
    public void setDay(int d) {...}  
    public void setMonth(int m) {...}  
    public void setYear(int y) {...}  
}
```

Madhavan Mukund Java modules Programming Concepts using Java 5/8

So, a small update also about these accessor and mutator method. So, remember that these are methods which are public which we use to query and update the private instance variables. So, here is a typical case. So, we have a date class which has three quantities day month and year. So, we have these accessor variables which get the current values of the day month and year and we have these mutated values which allow us to set the day month and year.

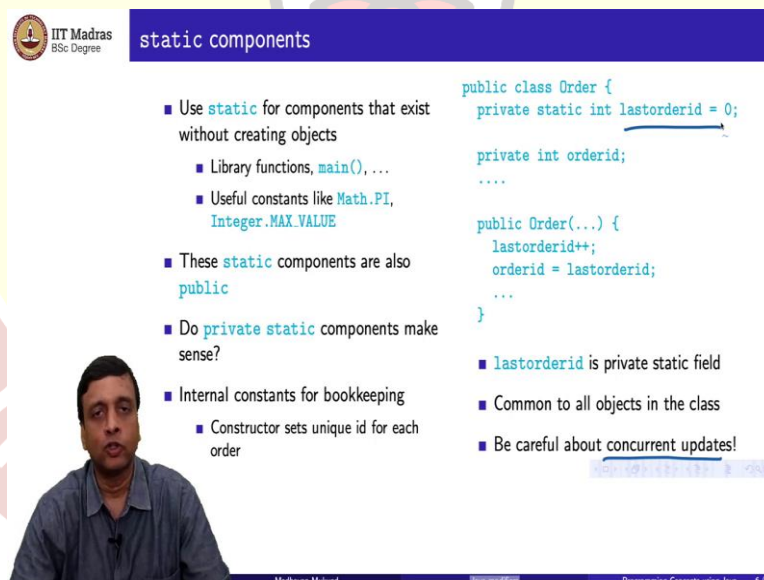
So, one difficulty with this particular implementation is that we independently set the day month and year but we know from our knowledge of the calendar that this is not completely safe right. Because I could now set something to be not just an illegal date like the 32nd of a month but there are different months that have different number of days. So, if I have a month with 30 days like June then I should not try to set the date to 31 which is legal in other context but not in this context right.

So, there is a kind of interdependence between these three values which the update should respect and this update in respecting this update is difficult if we are doing these updates one at a time right. So, we should not allow the user to actually separately set this similarly for instance February of 29th of February is normally not allowed but if it is a leap year it is allowed.

So, in that case we need all three quantities not just the day and the month we need to know the year is a leap year in that case 29th of February is a legal date. So, it would be better to replace this by a single function which allows us to set the date as a whole where the user has to provide all three values and if the user provides all three values then we can validate this combination before we update it.

So, when we design our accessor and mutator methods we also need to make sure that we are not blindly just allowing updates of individual fields but also maintaining some kind of internal consistency that we require between these fields.

(Refer Slide Time: 09:49)



IIT Madras
BSc Degree

static components

- Use **static** for components that exist without creating objects
 - Library functions, `main()`, ...
 - Useful constants like `Math.PI`, `Integer.MAX_VALUE`
- These **static** components are also **public**
- Do **private static** components make sense?
- Internal constants for bookkeeping
 - Constructor sets unique id for each order

```
public class Order {  
    private static int lastorderid = 0;  
  
    private int orderId;  
    ....  
  
    public Order(...) {  
        lastorderid++;  
        orderId = lastorderid;  
        ...  
    }  
}
```

- `lastorderid` is private static field
- Common to all objects in the class
- Be careful about concurrent updates!

Madhavan Mukund Java modifiers Programming Concepts using Java 6 / 8

So, let us move to this other keyword static. So, as we said we use static for components that need to exist before we create any objects. So, useful a typical example of this is this initial function that we want to call main. So, obviously before the java program starts executing there is no object right. So, we cannot expect main to live in the context of an object. So, main must be a static method but we also said that we have a very large number of library functions.

So, we have all the math functions then we have functions like sorting arrays and so on. And we do not want to have to create an array object in order to access this or create a math object to access these. So, library functions are also typically static functions. The other reason we use this is for instance variables. So, for example the math library defines a constant pi which is equal to the mathematical value of pi which we can use.

Similarly if we want to know the current largest integer that we can use there is something called max value in this class integer which tells us this. So, these are now numbers. So, these are instance variables but these are numbers which exist without us having to create a class. So, of course one important feature of all these is these are public right we can use we can call these functions we can access these values and so on.

So, the next question is why should or could one use something which is static but private right. So, something that is static means it exists without the class having to create an object at the same time it is private. So, it can only be used within that class. So, where would one do such a thing. So, here is a typical example. So, supposing when we are processing orders every time we create a new order we have to assign to it a unique order id.

Now this is not specific to orders you could be doing enrollment for example. So, students may be applying for something. So, each person has to get a new enrollment number right. So, in many contexts where you have to assign unique identifiers to new objects you have to keep track of the identifiers which have been used so far. So, here is a simple way to do it you create a static integer which stores the last order id which has been used.

So, far so now in the constructor. So, this is the constructor when I create so each order has its own private order id which is associated with that order. So, when I construct an order the first thing I do is I increment this last order id. So, that I get a new one and then i set my order id to be this one. So, as a result of this at any given point that last order id points to the last one that has been used.

So, if you increment it by one assuming of course you do not overflow beyond the largest integer that it can store if you keep incrementing it each new order is guaranteed

to get a new order id. So, this being a private static field what it means is that it is independent of the objects. So, in practice what it means is that this value is created when the class is defined and it is shared by all the objects.

So, all the objects have access to the same value last order id it is a single memory location. So, whenever any object in this class tries to query it or update it it will work at the same time it is private. So, it is only available within this class. So, only objects of this class can look it up and update it right. So, it is not available outside the class but it is a useful thing to keep track of dependencies between objects in the class.

So, this is one example where you want to create an order id there are other examples you can imagine one example is to keep track of the number of objects for instance you can just. So, this is implicitly doing that every time you create an object you are incrementing the order id. So, you are also keeping track of how many objects were created. But you can also do this for function calls you could have individual methods and you could keep track with a counter a static counter which tells you across all the objects how many times a given function was called right.

For a given object it can keep it keep track of it inside itself but for all the objects if you want to keep track across all objects of a given type how many times a function was called then you need a counter which is static. You can use this for instance to see which functions are used more often than other functions some kind of code profiling as it is called. Now one thing to keep in mind when we have this kind of shared update, right.

So, remember this last order id is being updated by all objects of this class. So, what if you have a system which is say a web based system where people can create an order. So, supposing it is an online shopping cart right online shopping site. So, there might be two users in different locations who are trying to order something at the same time. So, they are completely independent users there is no correlation between them in terms of what speed they work at and then in what time sequence they work.

So, if both of them try to create an order it could be there could be a situation where both of them look up the current last order and both incremented to the same value. So, this is a problem that we will look at in much more detail as we get to the second half of this

course. But one has to be very careful when we have shared updates like this we have to be very careful about what happens when these updates are potentially concurrent.

When two or more different objects can have access to this value and update it there should not be any inconsistencies it is like if two people are trying to book the last seat in a plane or the last berth on a train. Then when they log in they may both see that the seat is available but at the end of it, it should not be allotted to both of them because there will be a conflict otherwise.

(Refer Slide Time: 15:20)

IIT Madras
BSc Degree

final components

- `final` denotes that a value cannot be updated
- Usually used for constants (`public` and `static` instance variables)
 - `Math.PI`, `Integer.MAX_VALUE`
- What would `final` mean for a method?
 - Cannot redefine functions at run-time, unlike Python!
- Recall **overriding**
 - Subclass redefines a method available with the same signature in the parent class
- A `final` method cannot be overridden

Madhavan Mukund Java modifiers Programming Concepts using Java 7/8

The other keyword or modifier that we have seen is final. So, final as we said is Java's way of saying that something is fixed it cannot be updated. So, we typically use it in a public static form for these kinds of constants. So, we do not just want `math.pi` to exist independent of an object being created we also want it to be the same value and we should not allow the user to change it.

The same with this `max value` in the class `integer`. So, these are field which exist statically that is you do not need to create an object but also if you try to reassign them because they are final the java compiler will flag an error. So, this ensures that you never change the value of these things accidentally in your code. So, these apply for variables. So, what would it mean for a method right.

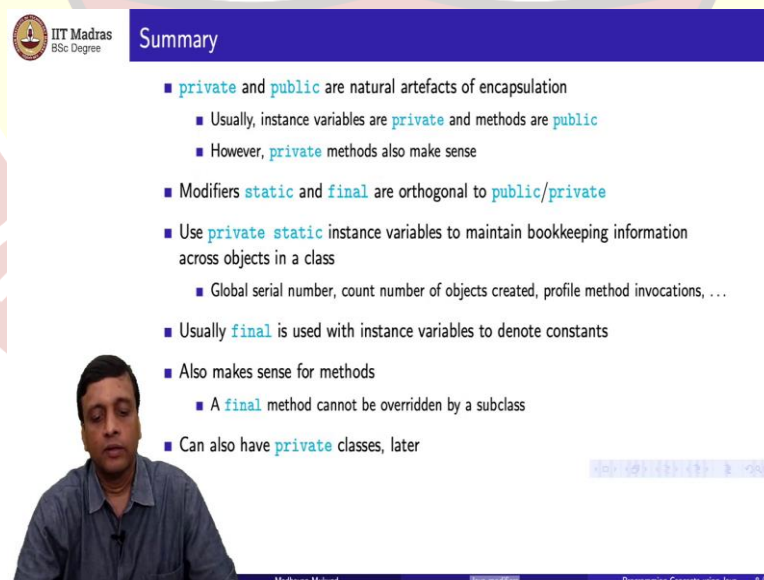
So, supposing I say that a method is final then presumably it says its value cannot be updated. Now in a language like python as you are running along in your code at some

point you can introduce a def right. So, you could have a definition of a function here and then you can redefine it and then henceforth python will use a new definition. So, in a language like python it is possible to redefine a function as you go along but in a compiled language like java the definition of a function is fixed.

When you compile the code you cannot redefine the same function later on. However there is another way in which we redefine which is overriding right. So, if I have a class a and a class b below it. So, if I have defined an f here and if I define another f with the same signature the same input and output parameters then b is allowed to change a as a sub type of a.

So, if I make f to be a final function in a then this is not allowed. So, this is one way of controlling that subtypes cannot redefine a method in a parent type. So, this is one way of controlling what happens when you subtype. So, that is a kind of not obvious use of final. So, final for a constant makes sense final for a function does not prima facie makes sense because you cannot update the value of a function in a language like java but because of overriding through this subtype mechanism it is something that we need to be able to control.

(Refer Slide Time: 17:42)



IIT Madras
BSc Degree

Summary

- **private** and **public** are natural artefacts of encapsulation
 - Usually, instance variables are **private** and methods are **public**
 - However, **private** methods also make sense
- Modifiers **static** and **final** are orthogonal to **public/private**
- Use **private static** instance variables to maintain bookkeeping information across objects in a class
 - Global serial number, count number of objects created, profile method invocations, ...
- Usually **final** is used with instance variables to denote constants
- Also makes sense for methods
 - A **final** method cannot be overridden by a subclass
- Can also have **private** classes, later

Mathavan Mahand Slide modifiers Programming Concepts using Java 8 / 8

So, to summarize we have a lot of modifiers in java which arise because of the complexities involved with object oriented programming. So, private and public are natural consequences of having encapsulation and you usually want your variables to be private your functions to be public. But as we saw sometimes it is useful to have some

internal methods inside a class which allow us to do something like helper like extending a stack. So, private methods do make sense.

So, private and public these are all different things they you can combine them in different ways. So, you can take private and public and combine it with static and combine it with final and these all are in different dimensions. So, to speak they do not interfere with. So, something is either private or public but having declared it private or public it can be static or not static it can be final or not final.

So, private static instance variables are useful for some kind of book keeping within a class for objects across that class. So, we saw this example where we wanted to maintain some kind of a global serial number across all the objects. So, we wanted to assign an order id which is distinct for every order but as we mentioned we could also be keeping track of function invocations and trying to see for instance how many times across all objects a given function is called and this could help us with this kind of profiling to find out how a particular class is being used by other classes.

So, similarly final is usually used for constants right to declare constant values that should not be changed like pi or maximum value in integer and so on but in a context of a method it means that this method cannot be overridden by a sub class. So, this is one way of controlling what happens when we do sub classing. Now these modifiers can also be assigned to classes.

So, we have talked mainly in this unit about modifiers attached to methods and to instance variables. So, for instance what would a private class mean right. So, we will it is possible to have private classes and we will talk about this a little later.